

RADAR URBANO

RADAR URBANO · BÍBLIA

CAPÍTULO I · VISÃO E ORGANOGRAMA

I.1 O que é "a máquina"

O que é: Cinco agentes (Cartógrafo, Editor, Anfitrião, Tesoureiro, Redator-chefe), um Guardião humano, quatro conectores e o Render & Delivery; onze entregas; o dono só recebe. **Antes:** Nada. **Passos:** 1) Entregas 01 a 11, numeração fixa; 01 e 02 uma vez. 2) Cinco camadas; cada uma fala só com a de baixo; Gemini obrigatório para dado e Google. 3) Tenant = apartamento do cliente (I.4); zona = bairro com duas vagas por segmento (VII.2). **Deu certo quando:** Você explica tudo com "onze pratos, cinco cozinheiros, um chefe que prova, duas mesas por ramo por bairro". **Se deu errado:** Mais de cinco agentes ou onze entregas, ou o dono dentro da cozinha: desenho errado.

I.2 O que é um conector

O que é: Programa que só fala com um sistema de fora; não pensa; devolve comprovante (id + print) e custo. **Antes:** Nada; para ligar, capítulo III. **Passos:** 1) Agente nunca chama `googleapis.com` ou `graph.facebook.com`; só o verbo. 2) Verbos: `gbp.`, `places.search_text`, `wa.send_`, `tts.synthesize`, `render.*`, `delivery.send`. 3) Falha lá fora: erro claro e retentativa (I.5). Todo conector tem sandbox (V.3). **Deu certo quando:** Todo código que toca sistema de fora está em `connectors/`. **Se deu errado:** Verbo com dados de outros estabelecimentos: recuse. API mudou: conserto só em `connectors/`.

I.3 O que é um agente (e o ADK)

O que é: Programa com missão, cérebro via LLMGateway e ferramentas, rodando como job. **Antes:** V.1; esqueletos no capítulo V do Mapa Mundi. **Passos:** 1) Pasta por agente: prompt versionado, ferramentas, cérebro pelo LLMGateway. 2) Gatilhos: Cartógrafo Dia 0, Dia 7, sábado 03h, dia 1 02h · Editor segunda 04h, diário 05h · Anfitrião 30 min · Tesoureiro hora, 03h · Redator-chefe segunda 06h, dia 1 06h. 3) Uma missão por agente; até o P1 nada sai sem `guardiao.submit` ; última ação, `diario.log` . **Deu certo quando:** Existem `agents/cartografo` , `editor` , `anfitriao` , `tesoureiro` , `redator_chefe` . **Se deu errado:** Agente que faz tudo, fica esperando ou chama Google direto: erro. Sexto agente: cabe num dos cinco.

I.4 O que é "tenant"

O que é: O apartamento de cada cliente: todo dado tem `tenant_id` ; a ficha tem `segmento` e `zona` , que fazem a trava territorial ser código. **Antes:** Firestore (IV.3); zona verificada (VII.2). **Passos:** 1) `tenant_id` em toda tabela, fila, segredo e pasta; nasce só após `zone.check` . 2) Ficha: `nome` , `segmento` , `zona` , `tom_json` , `limites_json` , `termos[]` , `raio_m` , `dono_wa_hash` ; estados `created` → `billing_active` → `profiled` → `gbp_linked` → `live` . 3) `zones(segmento, zona, cidade, tenants_ativos, limite=2) : live` soma 1; offboarding subtrai. **Deu certo quando:** `tenant_id` na primeira coluna de toda tabela; nenhuma linha de `zones` acima de 2. **Se deu errado:** Três do mesmo segmento numa zona: congele o terceiro, avise o Head.

I.5 O que é "job idempotente"

O que é: Rodar duas vezes dá o mesmo que uma. **Antes:** `deliveries` e `actions_log` . **Passos:** 1) Etiqueta única: cliente + agente + entrega + período (`padaria|redator|boletim|2026-09-14`). 2) Antes: "já feita?" Se sim, para;

depois, grava em `deliveries` ou `actions_log` . 3) Ações no mundo conferem lá fora (avaliação já respondida?). Data de Brasília. **Deu certo quando:** O mesmo job duas vezes gera uma entrega, uma linha, um custo. **Se deu errado:** Duplicada: etiqueta não gravada. Data de domingo: UTC.

I.6 O que é o Diário de Bordo

O que é: `actions_log` , append-only, com print: hora, agente, entrega, ação, alvo, print, versão do prompt, modelo, custo. **Antes:** BigQuery; Storage para prints. **Passos:** 1) Última ação de toda publicação, resposta, envio ou edição: `diario.log` . 2) Print da tela pública; texto de terceiros só como hash e temas; o Sim/Não do dono vira linha. 3) Só inserção; erro vira linha de correção; job diário acusa ação sem print. Retenção 5 anos. **Deu certo quando:** Qualquer post tem a linha com print; qualquer avaliação mostra os minutos até a resposta. **Se deu errado:** Sem `prompt-versao` : agente fora do LLMGateway. Custo em branco: o alerta de R\$ 25 fica cego.

CAPÍTULO II · OS CÉREBROS

II.1 O que é um "cérebro" e como se contrata

O que é: Modelo de IA pago por token. **Antes:** Cartão e e-mail da empresa; faturamento (III.1). **Passos:** 1) Gemini pela Vertex AI (II.2), sem chave; Claude opcional (II.3). 2) `EXTRACT`, `MEASURE`, `CLASSIFY`, `GOOGLE_DECISION`, `AUDIO_IN`, `VISION` sempre Gemini; escrita usa `LLM_LANGUAGE_PRIMARY` e `FALLBACK`. 3) Modelos só em variáveis; Flash para volume, Pro para análise; Gemini R\$ 3–5, linguagem R\$ 2–4 por cliente/mês. **Deu certo quando:** Studio responde "OK" e `llm_calls` tem a linha com custo. **Se deu errado:** "billing not enabled": III.1. Claude medindo ou classificando: política furada.

II.2 Como ligar o Gemini (Vertex AI)

O que é: Vertex AI (faturamento da empresa, região, Grounding com Google Maps, cache de contexto). **Antes:** `radar-hml` com faturamento; papel de editor. **Passos:** 1) Console → **Vertex AI** → **Ativar API** → **Model Garden** → **Gemini Flash** → **Vertex AI Studio** → "Responda só: OK". 2) **Grounding:** confira **Google Maps** como fonte; **IAM:** Vertex AI User à conta de serviço. 3) `gcloud auth application-default login`; `pip install google-genai`; `genai.Client(vertexai=True, project="radar-hml", location="southamerica-east1")`. **Deu certo quando:** Studio "OK", grounding cita o Maps, script imprime "OK". **Se deu errado:** "Permission denied": papel ou login. "Model not found in region": `us-central1` e registre.

II.3 Como contratar o Claude (opcional)

O que é: Cérebro de linguagem só para escrever; nunca mede, classifica ou decide sobre o Google. **Antes:** E-mail e cartão da empresa; Secret Manager (IV.4). **Passos:** 1) `console.anthropic.com` → **Billing:** cartão e limite mensal (US\$ 50). 2) **API Keys** → **Create Key** `radar-hml` ; guarde como `llm-claude-key` ; feche a aba. 3) Sonnet em `LLM_LANGUAGE_PRIMARY` , Gemini Pro em `FALLBACK` (ou o inverso, por II.6); teste "Responda só: OK". **Deu certo quando:** Script imprime "OK"; o LLMGateway só o oferece para escrita. **Se deu errado:** "Invalid API key": espaço no fim. Limite atingido: veja `llm_calls` antes de subir.

II.4 Saída estruturada (JSON)

O que é: O cérebro preenche um formulário (`responseSchema`); "segundo" em vez de `2` quebraria o Mapa de Domínio. **Antes:** Nada. **Passos:** 1) Toda chamada leva `schema` ; o LLMGateway o coloca no `generateContent` . 2) Cartógrafo `{termo, ponto, posicao|null}` ; Anfitrião `{resposta, tema, sentimento, risco, suspeita}` ; Editor `{campo, situacao, sugestao}` . 3) Fora do formulário: repete com temperatura menor; duas recusas, falha alto; dados com temperatura zero. **Deu certo quando:** Em `llm_calls` a resposta é JSON válido só com os campos do esquema. **Se deu errado:** Muitas recusas: simplifique. Número como texto: tipo não declarado.

II.5 Prompt de sistema

O que é: Folha de instruções fixa de cada agente, versionada; uma frase muda todos os clientes. **Antes:** Esqueletos do capítulo V do Mapa Mundi; ficha do tenant. **Passos:** 1) Blocos: identidade, permissões, proibições,

formato; chaves {casa} , {segmento} , {zona} , {tom} , {servicos} , {horarios} , {limites} . 2) Proibições: prometer posição; brinde por avaliação; inventar serviço, preço ou horário; termo de busca no nome; dado do avaliador; gerar foto. 3) prompts/<agente>/v1.2.md ; mudou, v1.3 ; o Diário grava a versão; trocar em produção exige ll.6 ou sandbox. **Deu certo quando:** Versão na configuração igual à de uma linha recente do Diário. **Se deu errado:** Mudou "do nada": volte pelo Git. Nome de outro cliente: bug de tenant, congele.

II.6 Avaliação cega mensal

O que é: 100 exemplos reais por agente que escreve, gerados pelos dois cérebros, embaralhados, notados por duas pessoas às cegas. **Antes:** Planilha (entrada, A, B, nota 1, nota 2; gabarito escondido); sandbox; régua 5 "publicaria sem tocar" a 1 "não pode sair". **Passos:** 1) Entradas do Diário do mês anterior: Anfitrião 100 avaliações e perguntas; Editor 100 posts; Redator-chefe Boletins e três frases. 2) Gere em sandbox com a mesma versão de prompt; embaralhe; notas sem conversa. 3) Maior média vira LLM_LANGUAGE_PRIMARY ; registre em avaliacoes/AAAA-MM.md ; notas 1–2 viram casos de teste. **Deu certo quando:** Arquivo por mês em avaliacoes/ e a variável em produção bate com ele. **Se deu errado:** Diferença < 0,2: empate, custo decide. Discordância ≥ 2 pontos: régua ruim.

CAPÍTULO III · OS CONECTORES

III.1 Google Cloud: conta, projeto, faturamento

O que é: Onde a máquina mora; projeto é a pasta do ambiente, faturamento é o cartão. **Antes:** Conta Google da empresa com 2FA; cartão, CNPJ; e-mail do engenheiro. **Passos:** 1) `console.cloud.google.com` → **Novo projeto** `radar-hml` e `radar-prd`; anote o **Número do projeto**; **Faturamento:** ligue os dois; **IAM:** engenheiro como Editor. 2) **Biblioteca:** Vertex AI, Places API (New), Cloud Run Admin, Cloud Build, BigQuery, Firestore, Secret Manager, Scheduler, Pub/Sub, Text-to-Speech, Identity Toolkit. 3) `gcloud init`; `gcloud config set run/region southamerica-east1`; **Orçamentos** R\$ 500/mês; exportação de faturamento ao BigQuery. **Deu certo quando:** Dois projetos com faturamento; onze APIs "Ativada". **Se deu errado:** "Sem permissão": conta pessoal. Cartão recusado: banco.

III.2 Pedir acesso às Business Profile APIs

O que é: Formulário do Google, 1–2 semanas; pedido do dia 1. **Antes:** Número do projeto; site com nome, atividade e e-mail no domínio; texto de uso (gestão de Perfis da Empresa de PMEs; acesso por papel de Gerente; sem senha, sem extração em massa). **Passos:** 1) `developers.google.com/my-business` →

Prerequisites / Request access; preencha; data em `acessos.md` . 2) Enquanto espera: III.3, III.4, III.6, III.7. 3) Aprovado: ativar Account Management, Business Information, Performance, Q&A e Google My Business API (v4); **Cotas** > 0; teste `accounts.list` . **Deu certo quando:** Cinco APIs ativas, cotas > 0, `accounts.list` sem 403. **Se deu errado:** 15 dias sem resposta: reenvie e escreva no fórum oficial. Cota 0: outro projeto.

III.3 Conta operacional e OAuth

O que é: `operacao@radarurbano.com.br` é o funcionário que o cliente adiciona como Gerente; OAuth deixa o programa agir por ela sem senha. **Antes:** `radar-prd` ; conta no domínio; cofre de senhas para gente. **Passos:** 1) Crie a conta, 2FA, senha no cofre; **Tela de permissão OAuth:** Interno, ou Externo publicado (em Teste o token morre em 7 dias). 2) **Credenciais** → **ID do cliente OAuth** → **Aplicativo da Web** `radar-gbp` ; redirecionamento `http://localhost:8080/callback` ; segredo `gbp-oauth-client` . 3) `scripts/gbp_login.py` (`access_type=offline` , `prompt=consent` , escopo `business.manage`) só com a conta operacional; refresh token em `gbp-operator-refresh-token` ; rotação 90 dias. **Deu certo quando:** `accounts.list` devolve a conta e o token segue válido após 8 dias. **Se deu errado:** `redirect_uri_mismatch` : URL não idêntica. `access_denied` : conta errada.

III.4 Ser adicionado como Gerente no perfil do cliente

O que é: O cliente adiciona a conta operacional como Gerente: um minuto, minuto 5 a 8 do Dia 0. **Antes:** Cliente Proprietário, logado no celular com a conta dona (senão III.5). **Passos:** 1) Texto ao cliente: "Abra o *Google Maps* com a conta dona do perfil e pesquise o nome da empresa. *Três pontinhos* → *Configurações do Perfil da Empresa* → *Pessoas e acesso* (ou *Gerentes*) → *Adicionar* →

operacao@radarurbano.com.br → *Gerente* → *Convidar*." 2) Computador: `business.google.com` → **Configurações do Perfil da Empresa** → **Pessoas e acesso** → **Adicionar**. 3) `gbp.discover` aceita o convite, casa nome e endereço, grava `gbp_location_id` e `place_id`; `gbp_linked` dispara a entrega 01. **Deu certo quando:** `locations` preenchido, `gbp_linked`, Diagnóstico na conversa. **Se deu errado:** Não acha o menu: peça print. Não é proprietário: III.5.

III.5 Perfil não reivindicado ou não verificado

O que é: Sem reivindicar e verificar, ninguém edita. **Antes:** Cliente no estabelecimento, com e-mail e telefone da empresa. **Passos:** 1) Cliente pesquisa a empresa logado: "É proprietário desta empresa?" = A; "já foi reivindicado" = B; não aparece = C. 2) A: **Gerenciar agora** → verificação; B: **Solicitar acesso**, o dono atual tem 3 dias; C: `google.com/business` → **Adicionar sua empresa**, categoria principal com o Guardião. 3) Verificação: vídeo sem cortes (fachada com placa e número, interior, prova de que trabalha ali), até 5 dias úteis; verificado, III.4. **Deu certo quando:** `locations.status` sai de `PENDING_VERIFICATION`; `gbp_linked`; Dia 7 conta do vínculo. **Se deu errado:** Vídeo recusado: placa, número, loja aberta. Sem resposta: `support.google.com/business` com CNPJ.

III.6 Places API e chave restrita

O que é: Serviço pago que lista lugares para um termo perto de um ponto; o Cartógrafo mede posição com `places:searchText`. **Antes:** Faturamento; `curl`. **Passos:** 1) **Biblioteca** → **Places API (New)** → **Ativar**; **Chave de API** `places-prd` → **Restringir** → só Places API (New); depois IPs do Cloud NAT; segredo `places-api-key`. 2) Teste: `POST .../v1/places:searchText`, cabeçalhos `X-Goog-API-Key` e `X-Goog-FieldMask: places.id`, corpo `textQuery`, `pt-BR`, `BR`, `pageSize 20`, `locationBias.circle`. 3) Posição = índice do `place_id` (1–20) ou fora do top 20;

Maps Platform → **Cotas:** 5.000/dia. **Deu certo quando:** O teste devolve lugares; Métricas mostra "Text Search"; chave restrita. **Se deu errado:** "API key not valid": IP ou projeto. "REQUEST_DENIED": ativou a antiga.

III.7 Meta: portfólio, verificação, app e número da casa

O que é: Uma vez só: Portfólio de negócios, verificação da empresa, App com WhatsApp e um número da empresa: o número da casa. **Antes:** CNPJ em PDF, comprovante de endereço no nome legal, site, e-mail no domínio; conta pessoal de um sócio com 2FA; número novo que nunca teve WhatsApp; página de privacidade; cartão. **Passos:** 1) `business.facebook.com` → **Criar conta** "Radar Urbano" → **Central de Segurança** → **Verificação da empresa**. 2) `developers.facebook.com` → **Criar app** (Empresa) → **WhatsApp** → **Adicionar número:** nome "Radar Urbano", código por SMS ou ligação; anote `phone_number_id` (`DELIVERY_SENDER_PHONE_ID`) e `waba_id` ; **Básico:** `META_APP_ID/SECRET` ; modo **Ativo**. 3) **Usuários do sistema** `radar-system` com app e WABA; **Gerar token** (nunca expira) → `meta-system-user-token` ; **Webhooks:** URL `https://<wa-inbound>/webhooks/wa` , assine `messages` e `message_template_status_update` ; **Pagamento:** cartão. **Deu certo quando:** `hello_world` sai do número da casa como "Radar Urbano"; webhook recebe `sent/delivered/read` ; empresa verificada. **Se deu errado:** Verificação recusada: nomes não batem. "Número em uso": apague a conta no celular.

III.8 Modelos de mensagem para o dono (e o SAIR)

O que é: Fora da janela de 24 h só sai modelo aprovado. **Antes:** III.7; amostras de card, PDF e MP4; consentimento do Dia 0 registrado. **Passos:** 1) Gerenciador → **Modelos de mensagem** → **Criar**, Utilidade, pt_BR; corpos literais na seção 15 do brief (`{{1}}` nome, `{{2}}` conteúdo, `{{3}}` referência). 2) Janela aberta: card e áudio

livres; fechada: modelo com MP4; só status Aprovado. 3) SAIR: `wa_optout = true` , uma resposta, nada mais até novo Sim escrito; outro texto: transbordo humano. **Deu certo quando:** Cinco modelos Aprovados; o Sim volta pelo webhook; SAIR desliga na hora. **Se deu errado:** Rejeitado ou Marketing: tire tom de promoção. Vídeo recusado: MP4 H.264/AAC ≤ 16 MB.

III.9 Cloud Text-to-Speech: a voz da casa

O que é: Texto vira voz para o Boletim (≤ 60 s). **Antes:** Faturamento; roteiro de teste com números por extenso. **Passos:** 1) **Biblioteca** → **Cloud Text-to-Speech API** → **Ativar**; painel: Português (Brasil), **Chirp 3: HD**; escolha e anote em `TTS_VOICE_ID` . 2) Script `google-cloud-texttospeech` : `OGG_OPUS` , 48000 Hz, mono; envie ao seu celular por `wa.send_media` tipo `audio` . 3) 60 s $\approx$ 150 palavras; o conector recusa acima; números por extenso, sem SSML. **Deu certo quando:** Toca no WhatsApp como mensagem de voz, < 60 s. **Se deu errado:** Robótica: voz Standard/WaveNet. Chega como documento: OGG/Opus mono, `audio/ogg` .

III.10 Webhook: status e respostas do dono, e como testar

O que é: A Meta toca `wa-inbound` a cada evento: status (`sent` , `delivered` , `read` , `failed`) e respostas (Sim, Não, SAIR, texto, áudio). **Antes:** `wa-inbound` no Cloud Run ou `ngrok` ; `WA_WEBHOOK_VERIFY_TOKEN` , `META_APP_SECRET` . **Passos:** 1) Validação: `GET /webhooks/wa?hub.mode=subscribe&hub.verify_token=...&hub.challenge=N` → 200 com `N` em texto puro. 2) Recados: `POST` com `X-Hub-Signature-256` (HMAC do corpo bruto); publique em `wa.status` / `wa.inbound` ; 200 em < 2 s; idempotência pelo `wamid` . 3) Local: `ngrok http 8080` , cadastre a URL, **Verificar e salvar**, `hello_world` ; status → `deliveries` ; Sim/Não → `approvals.decided` ; "sair" → opt-out; outro texto → transbordo. **Deu certo quando:** Recados no log com 200 em < 2 s; `deliveries`

com `lido_em` ; um Sim em `approvals.decided` . **Se deu errado:** "Não foi possível validar": challenge com aspas. Nada no POST: WABA não assinada.

III.11 Link curto e QR de avaliação (o material do balcão)

O que é: Link oficial do perfil que abre a tela "escreva sua avaliação", virado em QR e PDF de balcão e comanda. **Antes:** Tenant `gbp_linked` e verificado; logotipo e tom; política do Google (sem brinde, sem "gostou?", sem tablet). **Passos:** 1) Dono: painel do perfil → **Receber mais avaliações**; máquina: `gbp.get_review_link` lê `metadata.newReviewUri` . 2) `render.qr` (correção de erro alta); `render.pdf` : cartão A6 e tira de comanda, "Avalie a sua experiência. Aponte a câmera para o código." 3) `entrega_radar` (documento) na primeira semana; em "Arquivos" no dashboard; linha ao dono: "não peça nota, não ofereça nada, não escolha para quem mostrar". **Deu certo quando:** QR lido fora da rede da loja abre a tela certa; a primeira avaliação por ele é respondida em < 2 h. **Se deu errado:** `newReviewUri` vazio: não verificado ou suspenso. "Página não encontrada": novo `place_id` .

CAPÍTULO IV · A FUNDAÇÃO

IV.1 Cloud Run

O que é: Programas sem servidor. **Antes:** `gcloud` ; repositório com `Dockerfile`. **Passos:** 1) `gcloud run deploy wa-inbound --source . --region southamerica-east1 --allow-unauthenticated`. 2) `gcloud run jobs create cartografo --source . --region southamerica-east1 --command python --args -m,agents.cartografo ; execute com --args --tenant=padaria`. 3) Conta de serviço por agente: Vertex AI User, BigQuery Data Editor, Firestore User, Secret Accessor só nos segredos dele. **Deu certo quando:** A URL responde e o job termina "Succeeded". **Se deu errado:** "Permission denied": falta papel. Build falhou: log do Cloud Build.

IV.2 BigQuery

O que é: Armazém de números; todo extrato, card e dashboard nasce de SQL aqui. **Antes:** Faturamento. **Passos:** 1) **Criar conjunto de dados** `radar`, `southamerica-east1` ; as 15 tabelas da seção 8 do brief via `sql/create_tables.sql` ; partição por data, cluster `tenant_id` ; expiração conforme IX.2. 2) `hours_equivalence` : post 20 · foto 10 · resposta a avaliação 6 · pergunta 4 · ajuste 10 · medição 0,5 por termo-ponto · auditoria 60 · Boletim 30 · extrato/PDF 45 · Mapa 60. 3) `SELECT COUNT(*) FROM radar.metrics_daily` → 0; cota diária de consulta (50 GB). **Deu certo quando:** 15 tabelas particionadas, com cluster e expiração visíveis. **Se deu errado:** "Not found: Dataset": região. Expiração "Nunca": console e Terraform.

IV.3 Firestore

O que é: Banco de fichas: um documento por tenant (configuração, estado, `wa_optout` , `editor_frozen`) e a fila `approvals` . **Antes:** Projeto criado.

Passos: 1) **Firestore** → **Criar banco** → Nativo → `southamerica-east1` . 2)

Coleção `tenants` , documento `padaria-teste` com os campos de l.4;

coleção `approvals` . 3) Regras: só contas de serviço; índices pelo link que o console sugere. **Deu certo quando:** O documento aparece e o programa o lê.

Se deu errado: "Insufficient permissions": Firestore User. Ficha sem `segmento / zona` : não passa de `created` .

IV.4 Secret Manager

O que é: Cofre de senhas de sistema; só contas autorizadas leem; troca sem tocar em código. **Antes:** API ativada. **Passos:** 1) **Criar segredo:** `places-api-key` , `gbp-oauth-client` , `gbp-operator-refresh-token` , `meta-app-secret` ,

`meta-system-user-token` , `wa-webhook-verify-token` ; por tenant `tenant-{id}-gbp-oauth` . 2) Segredo → **Permissões** → conta de serviço → Secret

Accessor. 3) Código lê `versions/latest` ; nunca em variável em claro; trocar:

Nova versão; rotação de 90 dias. **Deu certo quando:** O programa lê e

Registros mostra o acesso. **Se deu errado:** "Permission denied": Accessor faltando. Vazou: revoke na origem, versão nova.

IV.5 Cloud Scheduler e Pub/Sub

O que é: Relógio e quadro de recados; juntos, a cadência da seção 9 do brief.

Antes: Jobs no Cloud Run. **Passos:** 1) Tópicos: `wa.inbound` , `wa.status` , `gbp.review.new` , `gbp.question.new` , `editor.tasks` , `approvals.decided` , `deliveries.send` , `tenant.state.changed` , `cost.alert` , `zone.changed` ,

`agents.run` . 2) Scheduler, fuso `America/Sao_Paulo` : anfitrião `/30` · *visões* `0` · *coleta* `0 3` · horários `0 5` · *post* `0 4 1` · *Boletim* `0 6 1` , *envio* `0 7 1` · *reduzida* `0 3 6` · *dia* `1 0 2 1 *` . 3) Serviço `dispatcher` assina `agents.run` e dispara o job por tenant ativo. **Deu certo quando:** Na segunda seguinte o Boletim sai sozinho com `enviado_em` às 7h. **Se deu errado:** Horário errado: fuso UTC. Recado duplicado: normal (l.5).

IV.6 GitHub e deploy automático

O que é: PR aprovado em `main` vira revisão nova no Cloud Run pelo Cloud Build. **Antes:** Repositório privado `radar-urbano` . **Passos:** 1) Ramificações `main` (produção) e `hml` . 2) **Cloud Build** → **Gatilhos** → **Conectar repositório;** `^main$` com `cloudbuild.yaml` para `radar-prd` ; `^hml$` para `radar-hml` . 3) Ninguém envia direto para `main` ; PR revisado por outra pessoa. **Deu certo quando:** PR aprovado aparece em **Revisões** em minutos. **Se deu errado:** Build falhou: log do gatilho. Revisão ruim: volte à anterior.

IV.7 Terraform

O que é: Infraestrutura como texto (projetos, tabelas, tópicos, permissões, expirações). **Antes:** Engenheiro. **Passos:** 1) S1: `terraform apply` sobe os dois ambientes; se atrasar, console e `terraform import` até a S3. 2) Depois, todo recurso nasce no Terraform, inclusive retenção (IX.2). **Deu certo quando:** `terraform plan` em `radar-hml` sem diferença. **Se deu errado:** Diferença: alguém clicou no console; corrija no código.

CAPÍTULO V · OS AGENTES

V.1 ADK: primeiro agente em 20 minutos

O que é: Instalar o kit e rodar um agente de teste, molde dos cinco. **Antes:** Python 3.11+; login ADC; Vertex AI (II.2). **Passos:** 1) `python -m venv .venv && source .venv/bin/activate && pip install google-adk google-genai`. 2) `agents/ola/agent.py : Agent(name="ola", model=os.environ["GEMINI_MODEL_FLASH"], instruction="Responda em uma frase.", tools=[])`. 3) `GOOGLE_GENAI_USE_VERTEXAI=TRUE, GOOGLE_CLOUD_PROJECT=radar-hml, GOOGLE_CLOUD_LOCATION=southamerica-east1`; `adk web` em `agents/`, digite "oi". **Deu certo quando:** Responde no `adk web` e o log mostra a chamada ao Gemini. **Se deu errado:** `ModuleNotFoundError` : venv inativo. Região: troque a variável.

V.2 Dar uma ferramenta a um agente

O que é: Função do conector com nome e docstring clara; o ADK mostra ao cérebro, que decide quando chamar. **Antes:** V.1; GBP Connector com `create_post` em homologação. **Passos:** 1) `def create_post(tenant_id: str, texto: str, foto_uri: str) -> dict:` com docstring "Publica um post no perfil do cliente. Retorna id e print." 2) `tools=[create_post, diario_log, guardiao_submit]`; instrução: "monte o post, `guardiao_submit`; aprovado, `create_post` e `diario_log`". 3) `adk web` : "Publique o post da semana da padaria-teste". **Deu certo quando:** Post no perfil de teste e linha com print no Diário. **Se deu errado:** Não chama: docstring vaga. Argumentos errados: tipos e exemplos.

V.3 Sandbox

O que é: `RADAR_SANDBOX=1` troca as ferramentas que tocam o mundo por versões que só anotam; o cérebro roda, o mundo não. **Antes:** V.2. **Passos:** 1) Conectores em `dry_run=True` gravam em `actions_log` com `sandbox=true`; obrigatório em `radar-hm1`. 2) Perfil de teste no Google e número de teste da Meta para o que precisa tocar o mundo. 3) `scripts/seed_tenant.py` padaria-teste : um mês sintético; teste fixo: `zone.check` recusa a terceira na zona. **Deu certo quando:** Ponta a ponta em homologação sem linha com `sandbox=false`. **Se deu errado:** Ação real em homologação: rodou sem a variável; bloqueie por código.

V.4 Painel do Guardião

O que é: Fila de aprovação em lote. **Antes:** Google Sheets; depois Identity Platform. **Passos:** 1) Planilha Guardião : `id`, `tenant`, `agente`, `tipo`, `conteúdo`, `foto`, `risco`, `decisão`, `texto editado`, `decidido por`, `hora`; job a cada 5 min executa; abas QA e Zonas. 2) Verdade: login, prioridade, atalhos A/E/R, Firestore `approvals`, custo por tenant ao lado, transbordo, `zone.check`. 3) P0 tudo antes; P1 lote com meta > 85%; P2 direto com QA de 10%; dois olhos sempre para categoria, nome, endereço, notas 1–2. **Deu certo quando:** 100 itens sintéticos aprovados em < 90 min com registro. **Se deu errado:** Fila crescendo (SLA 4 h úteis): taxa de rejeição alta é prompt ruim, não falta de gente.

V.5 A grade de medição: raio, termos, pontos e a validação com 20 buscas

O que é: 3×3 a 5×5 pontos sobre o raio do cliente; por termo, uma `places.search_text` por ponto; posição = índice do `place_id`. **Antes:**

`place_id`; `termos[]` e `raio_m` do Dia 0; chave Places. **Passos:** 1) Raio: o que o dono disse; padrão 1.500 m em cidade, 3.000 m onde bairro não serve; 3×3 no Dia 0, Dia 7 e sábado, até 5×5 no dia 1; espaçamento = $2 \times \text{raio} \div (\text{lado} - 1)$, `radius` = espaçamento $\div 2$. 2) Até 25 termos; 3 a 5 termos-chave nas reduzidas; `rank_measurements(tenant_id, data, termo, grid_point, lat, lng, posicao)`, nulo = fora do top 20; verde top 3, cinza 4º–10º, vermelho fora do top 10. 3) Validação mensal: 20 buscas manuais no Maps do celular, conta neutra, localização no ponto; concordância $\geq 85\%$; rodapé do Mapa declara o método. **Deu certo quando:** Posição para cada termo-ponto; SVG com três cores; validação $\geq 85\%$. **Se deu errado:** $< 85\%$: `radius` grande ou termo genérico. Sempre fora do top 20: `place_id` errado.

CAPÍTULO VI · AS RECEITAS (BICICLETAS DAS ONZE ENTREGAS)

Bicicleta e máquina produzem o mesmo objeto, com a mesma cara. Ferramentas: app do Perfil da Empresa (Google Maps → foto → **Seu perfil comercial**), planilha por cliente, Canva, Google Docs, My Maps, WhatsApp Business no celular da casa. Toda ação gera print e linha na aba **Diário**. Passou de 7 h por cliente por mês: automatize essa entrega primeiro.

VI.01 Diagnóstico de Posição

O que é: Card + PDF do Dia 0: posição em um termo-chave nos 9 pontos, o que falta no perfil, três primeiras ações. **Antes:** Termo-chave e raio do Dia 0; **place_id**. **Passos:** 1) Enquanto o cliente adiciona o Gerente, busque o termo em 9 pontos do Maps (localização no ponto); anote. 2) App → **Editar perfil**: confira descrição, categorias, horários (e **Adicionar horário de feriado**), serviços, atributos, fotos. 3) Canva: 9 quadradinhos coloridos, lista do que falta, três frases; Docs → PDF; mande no minuto 8 a 10. **Deu certo quando:** O card chega antes dos 15 minutos. **Se deu errado:** Perfil pendente: só posição e o que é público; diga isso.

VI.02 Antes e Depois do Perfil

O que é: Card lado a lado no Dia 7 (desde **created**): cada campo antes e depois, mais a segunda medição. **Antes:** Prints do Dia 0; lista do VI.01. **Passos:** 1) Refaça a

busca nos 9 pontos. 2) **Editar perfil**: print de cada campo alterado (descrição, horários, serviços, fotos, posts). 3) Canva em duas colunas com a linha de posição e a frase do QR; WhatsApp até as 8h; arquivo no dashboard. **Deu certo quando**: Cada linha do card tem linha no Diário com print. **Se deu errado**: Perfil ligou depois: conte 7 dias do vínculo.

VI.03 Dashboard Radar

O que é: Link fixo com números do Google (com data de referência) e o que é próprio; hora em hora na máquina, diário na bicicleta. **Antes**: Planilha por cliente, "qualquer pessoa com o link". **Passos**: 1) Primeira linha: ligações pelo perfil, rotas, cliques no site, mensagens, impressões, data de referência; uma linha por dia. 2) Todo dia 7h: app → **Desempenho** → copie o último dia disponível (2–3 dias atrás) com a data. 3) Nota fixa: "ligações pelo perfil são toques no botão Ligar contados pelo Google"; peça para salvar na tela inicial. **Deu certo quando**: O cliente abre no celular e vê números com data. **Se deu errado**: Número sem data de referência: não publique.

VI.04 Boletim de Segunda

O que é: Áudio ≤ 60 s + card às 7h, cobrindo a semana fechada disponível. **Antes**: Planilha do VI.03; posição de sábado nos termos-chave; aba **Diário**. **Passos**: 1) Domingo: some a semana até a data de referência; avaliações novas e respondidas em **Avaliações**. 2) Segunda 6h: áudio ("Bom dia, {nome}", números por extenso, "dados do Google até quinta", ações, o que vem, "nada a fazer do seu lado"). 3) Card no Canva; 7h, áudio e card. **Deu certo quando**: 7h, ≤ 60 s, data de referência dita. **Se deu errado**: Google atrasado: diga "o Google ainda não liberou os números"; o resto sai.

VI.05 Mapa de Domínio

O que é: Imagem do dia 1 com a grade colorida por termo; sem concorrentes nomeados; rodapé com o método. **Antes:** Termos (na bicicleta, 5); Google My Maps. **Passos:** 1) 9 pontos do bairro, cada termo buscado no Maps com a localização no ponto; anote. 2) My Maps: marcador por ponto (verde/cinza/vermelho), camada por termo; exporte; rodapé "posição vista na busca do Maps em cada ponto; validação com 20 buscas". 3) Dia 1 até as 8h; link no dashboard. **Deu certo quando:** Todo ponto tem cor e o rodapé diz o método. **Se deu errado:** Não dá para ir aos pontos: modo anônimo com o endereço do ponto; anote.

VI.06 Diário de Bordo do Perfil

O que é: Tudo que foi feito no perfil, em português de gente, com print; linha no WhatsApp quando há ação relevante. **Antes:** Aba **Diário**; calendário de feriados nacionais e municipais. **Passos:** 1) Segunda: app → **Adicionar atualização** → foto real + texto; print. 2) 72 h antes de feriado: **Editar perfil** → **Horário de funcionamento** → **Adicionar horário de feriado** + post; print. 3) **Fotos; Descrição** (≤ 750 caracteres); serviços e atributos; categoria só com aprovação humana; linha no WhatsApp por ação relevante, 8h–19h. **Deu certo quando:** Toda ação tem linha com print; o resumo entra no Boletim. **Se deu errado:** Foto gerada ou de banco: nunca.

VI.07 Motor de Reputação

O que é: Toda avaliação respondida em ≤ 2 h; perguntas respondidas; QR; Boletim de Reputação mensal. **Antes:** Notificações do app; tom; planilha **Avaliações** (data, nota, temas, sentimento, respondida em). **Passos:** 1) **Avaliações** →

Responder, específica e humanizada; notas ≤ 2 por segunda pessoa antes;

Perguntas e respostas: < 4 h. 2) **Receber mais avaliações** → link → QR + PDF

(III.11) na primeira semana; suspeita: **Denunciar avaliação**. 3) Dia 1: card com nota, novas, respondidas, temas elogiados e de atenção. **Deu certo quando:** Nenhuma avaliação passa de 2 h (100% em 24 h); QR no balcão. **Se deu errado:** Dono quer convite por mensagem ou brinde: não existe neste produto.

VI.08 Extrato de Demanda

O que é: PDF do dia 1: contatos do mês e custo por contato (R\$ 499 ÷ contatos).

Antes: Desempenho do mês fechado. **Passos:** 1) Contatos = ligações pelo perfil + rotas + cliques no site + mensagens pelo perfil; impressões e palavras-chave só como contexto. 2) R\$ 499 ÷ contatos (exemplo padaria: 260, R\$ 1,92). 3) Docs → PDF de uma página com data de referência e a nota sobre "ligações pelo perfil"; WhatsApp até as 8h. **Deu certo quando:** Quatro linhas, soma, custo por contato, data. **Se deu errado:** Pediram "novos clientes" ou "receita": não entra; o dono coloca o valor dele por cima.

VI.09 Horas Devolvidas

O que é: Card do dia 1: ações do Diário × tabela de equivalência; tempo do dono = minutos dos Sim/Não. **Antes:** Aba **Diário**; tabela do IV.2. **Passos:** 1) Conte por tipo e multiplique (clínica: 8 posts, 6 fotos, 27 avaliações, 5 perguntas, 4 ajustes, 240 medições, auditoria, Boletins e extratos = 14h07). 2) Card com a soma, a tabela no rodapé e o tempo do dono ("2 minutos, em uma resposta de sim ou não"). **Deu certo quando:** Cada linha do card bate com o Diário. **Se deu errado:** Minuto sem linha no Diário: não conta.

VI.10 Voz do Cliente

O que é: Card do dia 1 com temas de avaliações e perguntas: três elogios, três atritos, uma frase real de cada, sem nome; alerta quando um tema dispara. **Antes:** Planilha **Avaliações** com temas. **Passos:** 1) Leia tudo do mês; marque tema e sentimento; conte. 2) Card: três mais elogiados, três de atenção, frase anônima, variação contra o mês anterior. 3) Tema que triplica na semana: linha no WhatsApp em horário comercial. **Deu certo quando:** Nenhum nome; cada tema com contagem. **Se deu errado:** Só avaliações e perguntas; conversas não existem aqui.

VI.11 Fechamento Executivo

O que é: PDF de uma página no dia 1 (e trimestral): cinco números com variação, três linhas do Extrato, horas, mapa em miniatura, o que fizemos, o que vem. **Antes:** VI.03, VI.05, VI.08, VI.09 prontos. **Passos:** 1) Modelo no Docs; números do mês e variação; mapa em miniatura; ações; próximo mês. 2) Trimestre: três meses lado a lado; PDF até as 8h; arquivo permanente no dashboard. **Deu certo quando:** Uma página, números conferidos contra o Extrato, sem promessa de resultado. **Se deu errado:** Cópia automática a terceiros: não existe.

CAPÍTULO VII · ONBOARDING

VII.1 O roteiro literal do Dia 0

O que é: Quatro mensagens, cada uma com um toque que vira estado. **Antes:** Zona verificada (VII.2); contrato por link (número do dono e aceite do canal) e pagamento; conta operacional; número da casa com modelos; relógio.

Passos: 1) Antes (5 min dele): contrato assinado → `zone.check` ocupa a vaga e cria o tenant (`created`); cartão → `billing_active` ; o dono abre a conversa pelo link do contrato (janela de 24 h); sem mensagem em 1 h, `pergunta_sim_nao` "Podemos começar?". 2) Minuto 0–5: cinco perguntas por áudio (o que mais vende e como o cliente procura; bairros ou raio; jeito de falar; o que pode fazer sem perguntar; número e como quer ser chamado); Gemini transcreve; confirmação por `pergunta_sim_nao` ; o Sim grava tudo (`profiled`) e vale como opt-in. 3) Minuto 5–8: texto do Gerente (III.4); `gbp.discover` liga; medição 3×3, auditoria, três frases, PDF. 4) Minuto 8–10: "Seu painel: {link}. Salve na tela inicial. Senha, se quiser, lá dentro; ninguém aqui vê. Seu Diagnóstico de Posição: {card}. A partir de agora você só recebe: segunda 7h o Boletim; Dia 7 o Antes e Depois; dia 1 o Mapa e os extratos."; abrir o link registra a entrega 01 (`live`); pare o relógio. **Deu certo quando:** `live` (ou pendente registrada); Diagnóstico com `lido_em` ; consentimento com data, hora e texto; ≤ 15 min. **Se deu errado:** Some: após 72 h, `aviso_radar` com o passo pendente, um por semana. Sem a pergunta 4: Editor em P0.

VII.2 Verificar a disponibilidade da zona (a trava territorial)

O que é: Antes de vender: nesta zona, neste segmento, já há dois? Zona = bairro do Google Maps (ou raio equivalente), no contrato com cidade e segmento. **Antes:** Endereço do perfil; categoria principal; `zones` e **Consultar zona** (ou planilha `Zonas : cidade , zona , segmento , tenants_ativos , limite , nomes , reserva , lista_de_espera`). **Passos:** 1) Zona: endereço no Maps, leia o bairro ("São Paulo · Jardim América"); onde bairro não serve, círculo com o raio da grade; segmento: categoria principal, teste "apareceriam na mesma busca?". 2) Livre: proposta com cidade, zona e segmento na cláusula; reserva por 5 dias úteis (conversa não reserva); assinado: `zone.check` em transação soma 1, publica `zone.changed` ; só então `created` . 3) Lotada: não venda, sem tenant provisório; lista de espera (nome, contato, data; ordem de chegada, ninguém fura); vaga abriu: comercial chama o primeiro em 2 dias úteis. 4) Mudou endereço ou categoria: nova verificação; sem vaga, mantém até o fim do ciclo pago e entra na lista da zona nova; zona redesenhada: quem está fica. **Deu certo quando:** Nenhuma linha de `zones` acima de 2, todo dia; todo `live` com zona e segmento do contrato; `zone.check` recusa a terceira em teste. **Se deu errado:** Vendeu em zona lotada: o tenant não nasce; Head devolve com a frente da lista. Bairro em disputa: vale o Maps.

CAPÍTULO VIII · OPERAÇÃO

VIII.1 QA por amostragem

O que é: Todo dia útil, 10% das ações publicadas ontem, lidas pelo print: `ok`, `editaria`, `errado`. **Antes:** Aba `QA`; `actions_log` de ontem; `qa_samples`; 20 min no início do turno. **Passos:** 1) `sql/qa_amostra.sql`: ontem, `sandbox = false`, só o que tocou o mundo; 10%, mínimo 20, máximo 60. 2) Leia como o cliente do cliente; `errado` com motivo; corrige no mesmo dia, com print e origem `qa`. 3) Metas: `errado` < 2%, `editaria` < 15%; dois dias fora, o agente volta ao P1 até revisar o prompt; dia 2, taxas por `prompt_versao` para II.6. **Deu certo quando:** ≥ 20 linhas por dia útil; taxas na meta; cada `errado` com correção. **Se deu errado:** Só `ok` por semanas: plante três erros num tenant de teste. Erros num tenant só: `tom_json`.

VIII.2 Custo por cliente (e o que fazer acima de R\$ 25)

O que é: Custo de tecnologia por tenant por mês; alerta em R\$ 25. **Antes:** `llm_calls`, `actions_log`, `rank_measurements`, `deliveries`; `sql/custo_por_tenant.sql`; `precos_unitarios`; faturas. **Passos:** 1) Dia 1 (Tesoureiro 05h): `llm_calls.custo` + buscas × preço Places + mensagens × preço utilidade + TTS + rateio de nuvem; confira com as faturas (10% de folga). 2) Referência: Gemini 3–5 · linguagem 2–4 · Places 3–8 · WhatsApp da casa 1–2 · TTS, Cloud Run, BigQuery, Storage 3–6 · total R\$ 12–25. 3) Acima: `cost.alert`; causas: grade ou termos demais (5×5 × 25 termos = 625 buscas), Pro onde Flash bastava, auditoria sem cache, job em dobro, loop; corrija na ordem barata; revise em 48 h. **Deu certo quando:** Média < R\$ 25;

nenhum tenant acima por dois meses; tabela batendo com as faturas. **Se deu errado:** Cartógrafo campeão: grade. Editor: sem cache.

VIII.3 Runbook: perfil suspenso

O que é: `locations.status = SUSPENDED` ; `tenant.state.changed` alerta o Head. **Antes:** Acesso Gerente; Diário de 30 dias e `profile_audits` ; comprovantes do cliente (CNPJ, fachada com placa, conta de luz). **Passos:** 1) `editor_frozen = true` ; Anfitrião lê, mas não responde; em até 1 h, pelo número da casa: o que houve, o que fizemos, o que vamos fazer, o que precisamos dele. 2) Audite 30 dias: termo de busca no nome, categoria trocada, endereço divergente, post fora da política; corrija com print; **Restabelecer perfil suspenso** com o Proprietário; número do caso na ficha. 3) Enquanto espera: Cartógrafo mede; Boletim diz "perfil suspenso desde {data}"; em 48 h, auditoria em 100% dos tenants; causa nossa vira prompt novo; restabelecido: descongele; `incidentes/AAAA-MM-DD-tenant.md` . **Deu certo quando:** Status ativo; causa registrada; auditoria dos 100% em 48 h; cliente avisado no início, a cada passo e no fim. **Se deu errado:** Recusado: comprovante fraco; reenvie. 14 dias sem resposta: Head assume. Quer cancelar: pode; ofereça congelar sem cobrar.

VIII.4 Runbook: API do Google parou

O que é: 429 (cota) ou 5xx em Business Profile, Places, Vertex AI ou TTS. **Antes:** **Logging** com `status >= 429` ; páginas de status; **Cotas**; conta operacional. **Passos:** 1) Qual API, qual código: 401/403 é credencial (III.3) ou Gerente removido (III.4); Google fora: 30 min sem mexer, não reinicie; 429: peça aumento, reduza cadência. 2) Avaliações fora > 90 min: Guardião responde à mão em `business.google.com` , Diário com origem `manual`

VIII.4 ; o Anfitrião reconhece depois; perguntas: meta 4 h; Performance API > 72 h: o Boletim diz. 3) Gemini fora: tarefas obrigatórias esperam (fila > 2 h alerta); linguagem usa fallback; Boletim sem verificação cruzada não sai: `aviso_radar` de atraso. **Deu certo quando:** Fila esvaziou sozinha; nada duplicado; nenhuma avaliação > 24 h nem pergunta > 4 h; `incidentes/AAAA-MM-DD-api.md`. **Se deu errado:** Fila não esvazia: 400 (API mudou) ou 401/403. Cota estoura todo dia 1: lotes (02h, 02h30, 03h).

VIII.5 Runbook: número da casa com qualidade baixa

O que é: A Meta rebaixou qualidade ou limite do número da casa. **Antes:** Gerenciador → **Números de telefone;** `deliveries` ; `wa.inbound` com SAIR. **Passos:** 1) Anote qualidade e limite; opt-outs em 30 dias ÷ donos ativos, > 2% é nosso; `deliveries` por modelo, dono e hora: `aviso_radar` > 1/semana, fora de horário (Boletim 7h segunda; dia 1 até 8h; o resto 8h–19h). 2) Pause `aviso_radar` por 7 dias; corrija hora e texto; SAIR respeitado na hora; cliente segue pelo dashboard. 3) Vermelho: **Solicitar revisão** (número único, só clientes pagantes, só serviço contratado, opt-out por uma palavra); faixa no dashboard; religue 25/50/100% em três dias. **Deu certo quando:** Verde; opt-outs < 2%; Boletim seguinte com `lido_em` na maioria; `incidentes/AAAA-MM-DD-numero-da-casa.md`. **Se deu errado:** Bloqueio permanente: segundo número na mesma WABA, modelos de novo, `DELIVERY_SENDER_PHONE_ID` novo; só com o Head. Queda sem opt-out: spam na primeira mensagem; confira nome de exibição.

VIII.6 Runbook: offboarding

O que é: O cliente cancelou; a máquina se retira, devolve tudo, apaga no prazo e libera a vaga. **Antes:** Cancelamento registrado e data de fim do ciclo;

ficha, segredos, Storage, BigQuery. **Passos:** 1) `closing` com a data; até lá tudo continua; no dia, `offboarding(tenant)` : Fechamento de encerramento com a série desde o Dia 0. 2) `business.google.com` → perfil → **Usuários** → conta operacional → **Remover**; `deliveries` fecha; ZIP com link assinado de 30 dias (Diário em CSV com prints, PDFs, cards, mapas, fotos, posts, respostas, métricas). 3) Desative `tenant-{id}-gbp-oauth` ; `zones.tenants_ativos` – 1 no mesmo dia; `zone.changed` ; comercial chama o primeiro da lista; `closed` com data; `offboarding/AAAA-MM-DD-tenant.md` .
Deu certo quando: Fechamento e ZIP entregues; conta fora de **Usuários**; segredos desativados; vaga livre e publicada; `closed` ; perfil intacto. **Se deu errado:** Quer voltar: `reopen(tenant)` dentro da retenção; acessos e vaga de novo. Conta não sai do perfil: registre, avise por escrito, desative o segredo.

CAPÍTULO IX · SEGURANÇA E LGPD

IX.1 Pedido de titular

O que é: Avaliador, autor de pergunta ou dono pede para ver ou apagar dados; 15 dias. **Antes:** Pedido por escrito (`radarurbano.com.br/privacidade`); link ou texto exato; do dono, mensagem do número cadastrado; `dsr(tenant, hash)` . **Passos:** 1) Identidade com o mínimo; `dsr` procura em `reviews.texto_hash` , `questions.texto_hash` , `tenants.dono_wa_hash` , `actions_log.alvo` ; ver: `modo='exportar'` → PDF. 2) Apagar terceiro: hash vira marcador, temas e sentimento limpos; ficam data, nota, `respondida_em` ; resposta pública apagável por `reviews.reply` , com aviso à empresa; prints ficam. 3) Apagar dono: offboarding; apague `dono_nome` , `dono_wa_hash` , `tom_json` , transcrição, áudios; `actions_log` fica; registre em `lgpd/pedidos/AAAA-MM-DD-tenant.md` ; avise o cliente (controlador). **Deu certo quando:** < 15 dias com registro; consulta pelo hash vazia; cliente avisado. **Se deu errado:** Não aponta a avaliação: sem texto não há hash. Quer apagar a resposta e a empresa não: decisão dela.

IX.2 Suboperadores e retenção

O que é: Quatro suboperadores: Google Cloud e APIs; Meta (só o número da casa); provedor do cérebro de linguagem; provedor de pagamento. **Antes:** `radarurbano.com.br/suboperadores` e `/privacidade` com data; BigQuery, Storage, Firestore; Terraform; jurídico trimestral. **Passos:** 1) Página com nome, país, finalidade e o que cada um recebe; data a cada mudança; uma linha aos clientes na entrega seguinte. 2) Retenção: `actions_log` e prints 5 anos ·

dados do tenant (`reviews` , `questions` , `rank_measurements` , métricas, `posts` , `deliveries` , arquivos) e do dono: contrato + 90 dias · respostas brutas da Places 30 dias · áudios do Dia 0 apagados após a confirmação por escrito · `llm_calls` e `qa_samples` 12 meses · ZIP 30 dias · contrato e nota fiscal pelo prazo da lei. 3) Expiração de partição no BigQuery; ciclo de vida por prefixo no Storage; job dos 90 dias após `closed` ; revisão trimestral em `lgpd/revisoes/AAAA-Tn.md` . **Deu certo quando:** Página com data; expiração em cada tabela; ciclo de vida em cada bucket; job dos 90 dias com log diário. **Se deu errado:** Tabela "Nunca": console e Terraform. Ferramenta nova recebendo texto sem passar por aqui: incidente; pare.

CAPÍTULO X · O PLANO DE MONTAGEM

X.1 O quadro das seis semanas

O que é: Seis colunas com "pronto quando" e uma linha vermelha no topo com as aprovações de terceiros (Business Profile APIs; Meta e número da casa; modelos): o caminho crítico. **Antes:** Quadro; capítulo X do Mapa Mundi; III.2 e III.7 lidos; engenheiro, Head, jurídico na S6. **Passos:** 1) Linha vermelha com data de envio e de resposta de cada pedido; reunião de 15 min em pé toda segunda, com evidência. 2) Sem Google: sandbox e bicicletas (S3 não depende); sem Meta: número de teste, 250 donos/dia bastam; S5: `zones` é a verdade antes de qualquer venda. 3) Quadro: S1 fundação e pedidos (`terraform apply` sobe os dois ambientes; três pedidos com data) · S2 conectores, LLMGateway, Diário, custo (busca devolve posição; toda chamada em `llm_calls`) · S3 Cartógrafo, Tesoureiro, dashboard, Diagnóstico (validação $\geq 85\%$; Diagnóstico em minutos após `gbp_linked`) · S4 Editor, Anfitrião, Guardião em planilha, Antes e Depois (tenant interno uma semana com print; 100 itens em < 90 min) · S5 Redator-chefe, TTS, Delivery, webhooks, extratos, Dia 0 e Dia 7 automáticos, `zone.check` (Boletim sintético às 7h; Dia 0 cronometrado; terceira empresa recusada) · S6 runbooks, SLOs, offboarding, jurídico, alerta de R\$ 25, piloto com 3 clientes por 30 dias em P0. **Deu certo quando:** 11 testes de aceitação passam; Dia 0 ≤ 15 min com três pessoas novas; piloto fecha um mês com as 11 entregas e custo $< R\$ 25$; ninguém tem senha de ninguém; nenhuma zona com mais de 2. **Se deu errado:** S4 sem Google: siga em sandbox. S5 sem Meta: 250 donos/dia.

GLOSSÁRIO

TERMO	DEFINIÇÃO
ADK	Kit do Google, em Python, para montar agentes.
Agente	Programa com missão que pensa com IA e usa ferramentas; cinco, mais um humano com painel.
Cache de contexto	Guardar no Gemini o que não muda para não pagar de novo.
Cérebro	Modelo de IA; Gemini para dado e Google, outro só para linguagem.
Conector	Programa que fala com um sistema de fora; quatro, mais o Render & Delivery.
Contato	Toque em Ligar, rota, clique no site ou mensagem pelo perfil, contados pelo Google. Nunca ligação atendida.
Custo por cliente	Gasto de tecnologia por tenant por mês; alerta em R\$ 25.
Custo por contato	R\$ 499 ÷ contatos do mês; a única conta feita para o cliente.
Data de referência	Último dia que os números do Google cobrem (2–3 dias atrás); acompanha todo número do Google.
Diagnóstico de Posição	Entrega 01: card e PDF do Dia 0 com posição atual, o que falta no perfil e as primeiras ações.
Diário de Bordo	<code>actions_log</code> : registro imutável de toda ação, com print.
Grade de medição	Pontos (3×3 a 5×5) sobre o raio do cliente onde se mede a posição por termo.
Grounding	Gemini consultando dados oficiais do Maps enquanto responde.
Guardião	Humano com painel: aprova em lote, audita por amostragem, dois olhos no que é de risco.
Idempotente	Fazer duas vezes dá o mesmo que uma.
Janela de 24 h	Após mensagem do dono, 24 h de resposta livre sem modelo.
Link curto de avaliação	Link oficial "Receber mais avaliações" do perfil, que vira QR e PDF de balcão.
Link fixo / mágico / único	Dashboard com token (vira ícone) / entrada sem senha / só para criar ou redefinir senha.

TERMO	DEFINIÇÃO
LLMGateway	Balcão único de cérebros: política, medição, <code>llm_calls</code> .
Número da casa	O único WhatsApp do Radar Urbano, na Meta, que fala só com o dono. SAIR interrompe.
Patamar (P0, P1, P2)	Tudo aprovado antes / lote com meta > 85% / publicação direta com QA de 10%.
Performance API	Porta do Perfil da Empresa que devolve métricas com atraso e data de referência.
Places API	Serviço pago do Google sobre lugares; busca por texto como proxy da lista local; só <code>place_id</code> além de 30 dias.
Ponto da grade	Coordenada onde se mede a posição: verde top 3, cinza 4º–10º, vermelho fora do top 10.
Prompt de sistema	Folha de instruções fixa e versionada de um agente.
Refresh token	Chave de longa duração do OAuth; o segredo mais importante do GBP Connector.
Sandbox	Modo de teste que não toca o mundo.
Segmento	Categoria principal do perfil e as buscas que ela disputa; com a zona, define a vaga.
Tenant	O apartamento de cada cliente: tudo que é dele, isolado.
Token (Meta)	Chave do usuário de sistema do Radar Urbano que autoriza o número da casa. Uma só.
WABA	Conta do WhatsApp Business do Radar Urbano na Meta, dona do número da casa. Uma só.
Webhook	Campainha: a Meta toca a URL da máquina quando algo acontece.
Zona	Bairro do endereço do cliente no Google Maps (ou raio equivalente) ; no máximo duas empresas do mesmo segmento.
zone.check	Conta e ocupa a vaga da zona em transação; recusa a terceira.